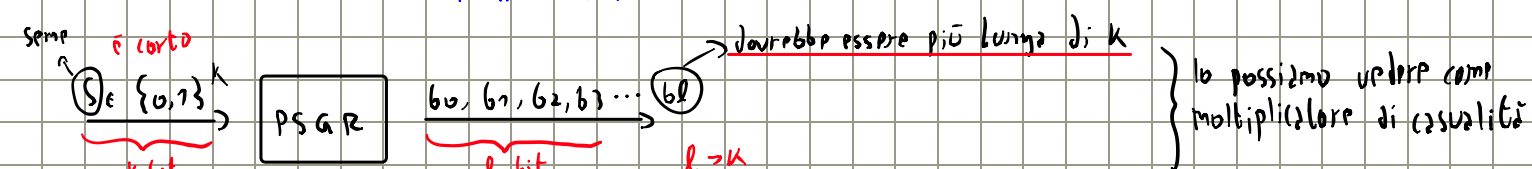


GENERAZIONE BIT PSEUDO CASUALI

Vogliamo generare una sequenza di bit che sia equivalente ad aver lanciato una moneta ogni volta. All'interno di un computer non c'è niente di casuale, perché è tutto fortemente deterministico. Il risultato della generazione di un bit casuale è imprevedibile da parte dell'osservatore, appare casuale. In realtà sono pseudocasuali, la generazione di bit pseudocasuali ha come origine un seme, da cui attraverso una serie di processi, genera una SEQUENZA DI BIT PSEUDO CASUALI. L'intento è quello di creare un GENERATORE che produce una sequenza di bit tale che nessuno in tempo polinomiale è in grado di capirlo.

↳ COS'È UN GENERATORE DI BIT CASUALI



// Avremo la stessa casualità con $l > k$, ma agli occhi di unologo per sembra più casuale

↳ COSA VUOL DIRE GENERARE BIT PSEUDOCASUALI?

- se ricevo $b, b_1, \dots, b_l$ faccio fatica ad indovinare b_{l+1} , ovvero lo si indovina con vantaggio piccolo, rispetto a quelli che cercano di indovinare senza sapere nulla.

$$\forall A \in \text{PPT}, \forall l \text{ la probabilità } \left(\Pr \left[s \in \{0,1\}^k, b_1 \dots b_l b_{l+1} \leftarrow G(s) \right); b \leftarrow A(b_1 \dots b_l); b = b_{l+1} - \frac{1}{2} \right] \right) < k^{-\omega(1)}$$

scegliere un seme di k bit
GENERATORE genera l bit
prova ad indovinare il prossimo bit con A
faccio il test

Appare casuale quando non so prevedere il prossimo lancio di moneta (bit successivo), ovvero quando la probabilità si avvicina alla stessa probabilità con cui qualcuno cerca di indovinare a caso. Quindi probabilità è più vicina ad $\frac{1}{2}$ più di qualsiasi altro polinomio

↳ PROVIMO A DARE UNA DEFINIZIONE DIVERSA

$\forall D \in \text{PPT}$ SIA

$$P_k^{D,R} = \Pr \left[b_1 \dots b_l \in_R \{0,1\}^l; D(b_1 \dots b_l) = 1 \right] \quad \text{// la probabilità che il DISCRIMINATORE restituisce 1 se prende in input bit casuali}$$

l bit veramente casuali

$$P_k^{D,G} = \Pr \left[s \in_R \{0,1\}^k; b_1 \dots b_l \leftarrow G(s); D(b_1 \dots b_l) = 1 \right] \quad \text{// bit pseudo casuali}$$

$$|P_k^{D,R} - P_k^{D,G}| < k^{-\omega(1)} \quad \text{// il } D \text{ riesce a vedere le differenze?}$$

Questa definizione è più forte, ma non ha un protocollo per implementarlo. Alla fine la seconda implica la prima, ma anche la prima implica la seconda. le 2 definizioni sono equivalenti

↳ FUNZIONA QUESTA DEFINIZIONE?

fissiamo Z_p^*, g

$$x_0 = s$$

$$x_{i+1} = g^{x_i} \quad s \quad g^s \quad g^{g^s} \quad g^{g^{g^s}} \dots g^{g^{g^{g^s}}} \} l$$

$$b_i = \begin{cases} 0 & \text{se } x_i < \frac{p-1}{2} \\ 1 & \text{altrimenti} \end{cases}$$

RET $b_l b_{l-1} \dots b_2 b_1 b_0$ // bit alla rovescia

↳ SUPPONIAMO PER ASSURDO CHE ESISTA (LA PRIMA DEFINIZIONE, PERCHÉ È PIÙ FACILE)

$$\begin{array}{ccccccc} s & g^{x_0} & g^{x_1} & \dots & g^{x_{l-1}} & g^{x_l} & \frac{1}{2} + k^{-c} \\ x_0 & x_1 & x_2 & \dots & x_l & \boxed{x_{l+1} \dots x_l} \\ b_0 & b_1 & b_2 & \dots & \boxed{b_2} & \boxed{b_{l+1} \dots b_l} \end{array} \leftarrow$$

// riesce a prevedere b_2 con un vantaggio più grande di qualche polinomio

$$x \quad g^x \quad g^{g^x} \quad \dots \quad g^{g^{g^x}} \quad g^{g^{g^{g^x}}} \quad g^{g^{g^{g^{g^x}}}}$$

RET $b_l b_{l-1} b_{l-2} \dots b_{l+1} b_2 \dots b_0$

Avendo x_{l+1} e cercando di indovinare b_2 , equivale a calcolare HCP del logaritmo discreto

$$b_2 = A(b_{l+1} \dots b_l) \text{ con } \Pr > \frac{1}{2} + k^{-c}$$

⚠ una funzione con bit restituiti a rovescio, non "fatti" perché il calcolo del bit successivo è semplice = niente il calcolo di una funzione one way

• la correttezza sta nel fatto che, anche conoscendo i semi, non si può calcolare b_2

↳ QUINDI?

Se $b_l \dots b_0$ soddisfa definizione 1 allora soddisfa la 2
 quindi $b_l \dots b_0$ è indistinguibile da sequenza casuale
 posso concludere che $b_0 \dots b_l$ indistinguibile da sequenza casuale?

Sia $D: \begin{matrix} b_l \dots b_0 \\ b_0 \dots b_l \end{matrix}$ restituire $D(b_0 \dots b_l)$

anche con i bit dritti funziona

↳ utilizziamo la **one-way**

f **one-way**

H **hard core predicate**

S	$f(s)$	$f(f(s)) \dots$	$x_0 = s$
x_0	x_1	$x_2 \dots$	$x_{i+1} = f(x_i)$
b_0	b_1	$b_2 \dots$	$b_i = H(x_i)$

se esistono la HCP della **one-way**, i generatori di bit pseudo casuali funzionano

↳ HCP dell'elevamento al quadrato

$\mathbb{Z}_n^*$
 $n = pq$

s
 $x_0 = s^2$
 $x_{i+1} = x_i^2$
 $b_i = \text{LSB}(x_i)$

s^2	$(s^2)^2$	$((s^2)^4)^2$
s^2	s^4	s^8
b_0	b_1	b_2

di quale radice quadrata, hanno tutte lo stesso uso?
non è invertita

↳ **merciamo su il crittosistema, Blum-Blum-Shoup**

Questo algo è basato sulla difficoltà del calcolo di radici quadrate in $\mathbb{Z}_n^*$, l'elevamento al quadrato in $\mathbb{Z}_{pq}^*$ è **one way trapdoor**, perché se conosciamo la fattorizzazione di n (p, q) allora sappiamo calcolare radici

• $\text{LSB}(\sqrt{x})$ è un **hard core predicate**

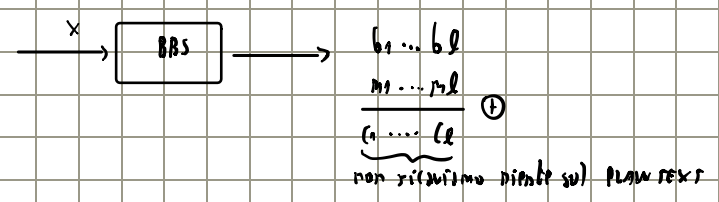
THEO. se $p, q \equiv 3 \pmod 4$ (**primi di Blum**)

allora per ogni quadrato x esiste un'unica radice che è un quadrato

prendiamo $n = pq$, $p_k = n$, $s_k = (p, q)$

ϵ $m = m_1 \dots m_l$

$x \in \mathbb{Z}_n^*$



non è sufficiente

↳ **COME FACCIAMO A DECIFRARE**

Si aggiunge $x^{(2^{l+1})}$ come informazione alla fine del cipher text quindi

$$c_1 \dots c_l \quad x^{(2^{l+1})}$$

ha la stessa complessità di **rsa** ed è estremamente sicuro

$$O(l^2)$$